Message passing: determinism, choice and buffer capacity

Block 1. Determinism, choice and buffer capacity in message passing

Once processes share nothing, where can the scheduler's choice still reach the result of a program, and what does the capacity of a channel change?

The topics of the two blocks and the lab

Block 1	The RC 4000 nucleus (1969); shared memory and the scheduler; processes that share nothing; Kahn process networks; contested communications and three resolutions; the RC 4000's two resolutions; channels (CSP, occam, Go); buffer capacity.
Block 2	Actors and mailboxes (Hewitt, Agha, Erlang); fairness; the delivery assumption and the boundary with session 9; the cost of message passing, and channels against actors.
The lab	A message-passing library built by the students: a toy RC 4000 nucleus, one queue under two disciplines, the handshake, `select`, and a closing comparison.

PART 1

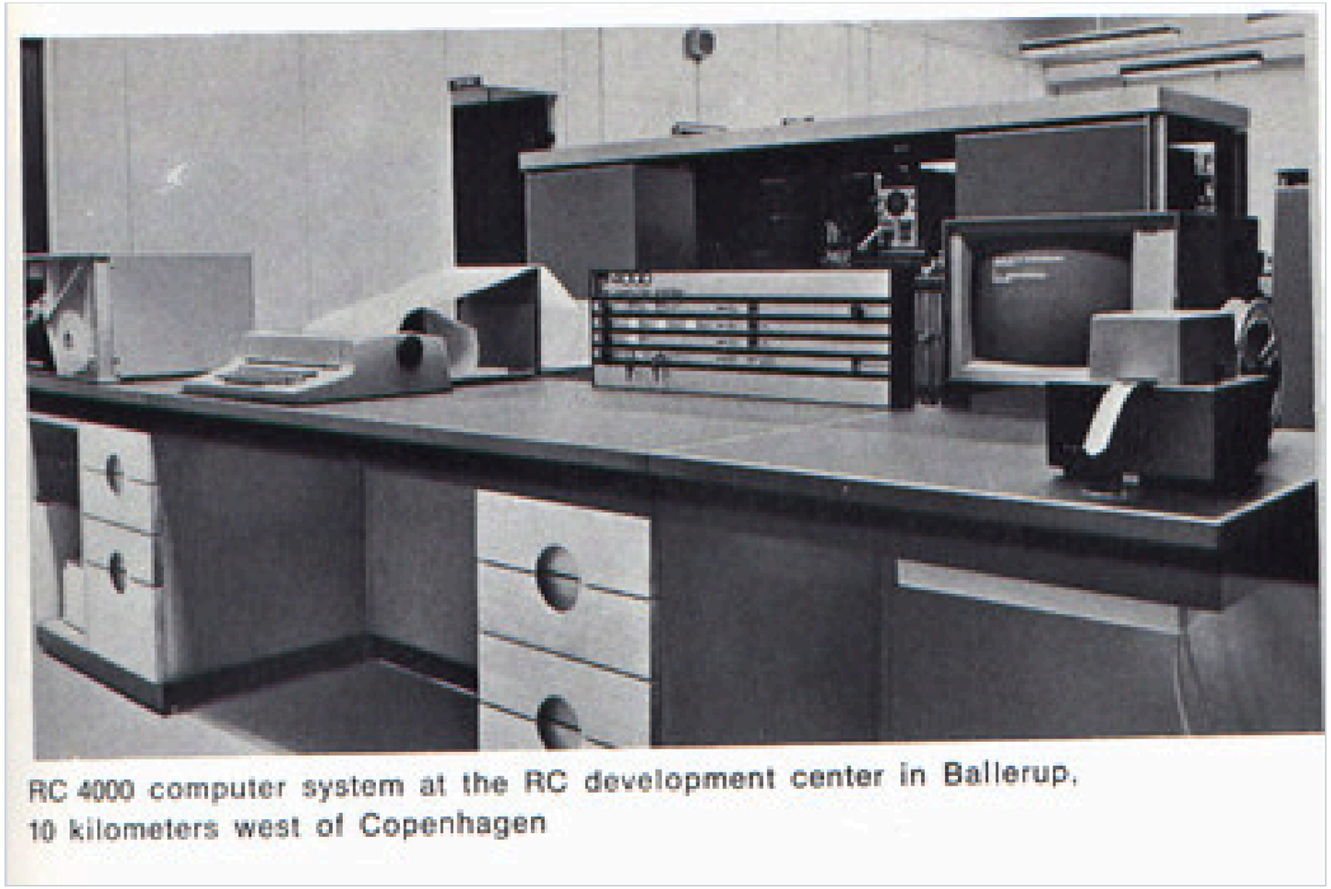
The RC 4000 nucleus, Copenhagen, 1969

the earliest message-based system of the session; messages chosen over semaphores

The RC 4000 and its nucleus

Regnecentralen's RC 4000 dates from 1967; its first prototype monitored an ammonia plant at Puławy in Poland.

In 1969 a group led by Per Brinch Hansen, aged 30, finished its multiprogramming system (Brinch Hansen, April 1970).



RC 4000 computer system at the RC development center in Ballerup, 10 kilometers west of Copenhagen. Regnecentralen, Highlights of RC's History (datamuseum.dk).

The RC 4000 and its nucleus

The group did not write a conventional operating system. It wrote a small **nucleus** with three jobs, in the paper's words:

1. the simulation of processes;
2. communication among processes;
3. the creation, control and removal of processes.

> Messages were its basic means of communication between processes. Operating systems and user programs ran on top as ordinary processes. The nucleus was 4,800 words; a send took 0.6 ms, a wait 0.4 ms, a full conversation about 2 ms.

Messages instead of semaphores

- Brinch Hansen had learned synchronization from Dijkstra's *Cooperating Sequential Processes*, the manuscript of 1965 that made semaphores widely known; the paper acknowledges it.
- Having learned semaphores, he withheld them from processes and gave them messages.

> **The semaphore concept alone "does not fulfill our requirements of safety and efficiency in a dynamic environment in which some processes may turn out to be black sheep and break the rules of the game" (page 239).**

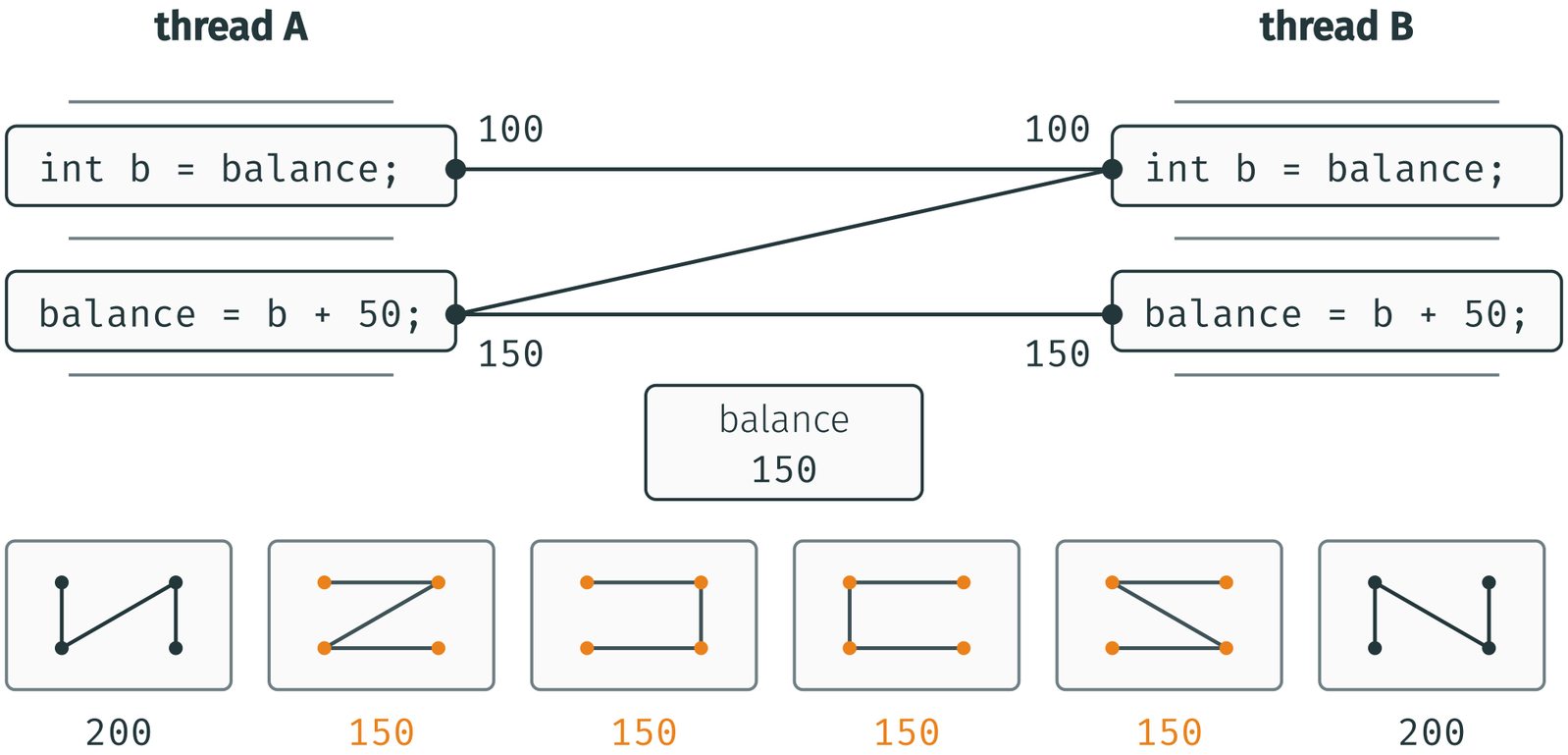
The machine returns twice in this block: for where its shared memory went (part 3), and for the two resolutions it uses at once (part 6).

PART 2

Shared memory and the scheduler

the lost deposit; determinism and nondeterminism; the data race; the memory models

The lost deposit



The marks between the instructions are the points where the scheduler may switch threads: all of them, since nothing in the text says which matter.

Thread B reads 100 too, before A has written.

Thread B writes 150 over it: A's deposit is lost.

One schedule: thread A reads 100.

Thread A writes 150.

All six schedules of the four instructions: two end at 200, four at 150. In the interleaving model, four of the six lose a deposit.

The lost update in a shared counter

The same loss happens to a counter of page views that many requests increment without a lock. Four Java threads each added one to a shared field 250,000 times.

```
$ java LostUpdate
plain field:    473,696   505,178   450,955
with a lock:  1,000,000 1,000,000 1,000,000
```

Three runs, below the 1,000,000 expected, and nothing reported an error; with a lock, or an `AtomicInteger`, the count was exact every time.

Every shared variable is a place where threads may compete. If two threads touch it, at least one writes, and nothing orders the two accesses, the result can depend on timing.

> What exactly is lost, and what is the name of the fault?

Determinism, nondeterminism and the data race

Deterministic program	Every run on the same input data gives the same observable results.
Nondeterministic program	The same input can give different results, depending on choices the program text does not fix, here the scheduler's. A clock or a random draw counts not as input but as a source of nondeterminism.
Execution and result	The **execution**, which steps run in which order, can differ from run to run while the **result** stays the same. Unless the execution is named, nondeterminism in this lecture means a difference in the result.
Data race	Two accesses to the same location, at least one of them a write, and nothing ordering them. Locks restore order, but only if every access takes them; two locks taken in opposite orders can deadlock.

The memory models and the interleaving model

The lost deposit is one interleaving of the two threads' steps, and the interleaving picture is a model, not the whole truth.

Language	What a program with a data race may do
C and C++	The program is undefined: anything can happen.
Java	A racy read returns a value some write produced, except a plain `long` or `double`, whose two halves may come from different writes; happens-before and causality still constrain which writes it may see.
Go	A read of a word-sized location sees some write to it; a race on a multiword value can give a torn value and corrupt memory, and the implementation may report the race and stop the program.

Every one of these models allows the interleavings and some allow more, so the lecture reasons in the interleaving model and says so.

Nondeterminism in a shared-memory program

The useful way to describe this is not "shared memory is hard".

> **In a shared-memory program, nondeterminism can enter at any conflicting, unordered access to shared data, and nothing in the program text marks which accesses those are. The scheduler makes the choice, everywhere, all the time.**

- It holds in a language whose types do not require shared data to be marked: Java, C and Go; Rust's types do (session 12).
- Hoare, 1978: processors that cooperate by "inspection and updating of a common store" can create severe problems for writing correct programs.

PART 3

Processes that share nothing

the first move of message passing; the shared memory inside the runtime

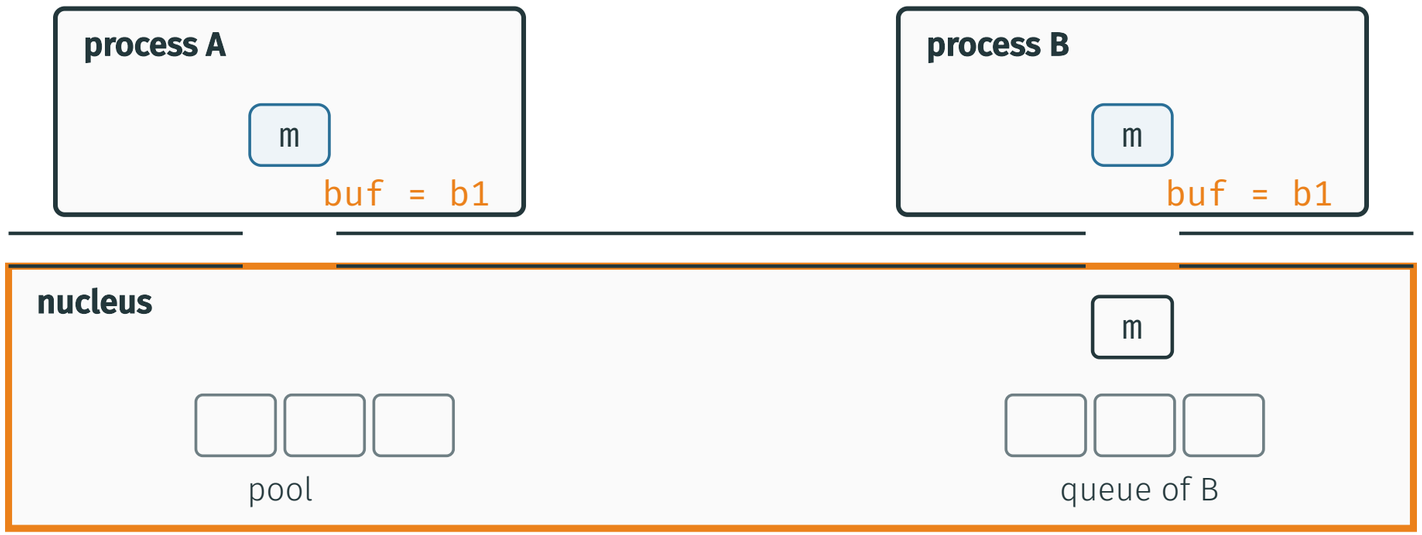
Message passing instead of shared variables

Each process owns its data, and the only way to affect another process is to send it a value: processes send and receive messages instead of reading and writing shared variables (Andrews and Schneider, 1983).

> The move fences nondeterminism into a few visible places. Inside a process that reads no clock, draws no random numbers, takes no other outside input and uses no construct whose order the language leaves open, code is sequential and deterministic. Timing can matter only where processes meet, at communications.

- What is sent is a value, not a pointer to memory both sides still use, which Go permits (session 12).
- Go's iteration over a map is one construct whose order is left open.

The send path of the RC 4000



In the model, m goes from A to B, and A keeps its own copy.

Send message: the nucleus takes a free buffer from the pool.

It links the buffer, b1, at the tail of B's queue, and send message returns its identity to A.

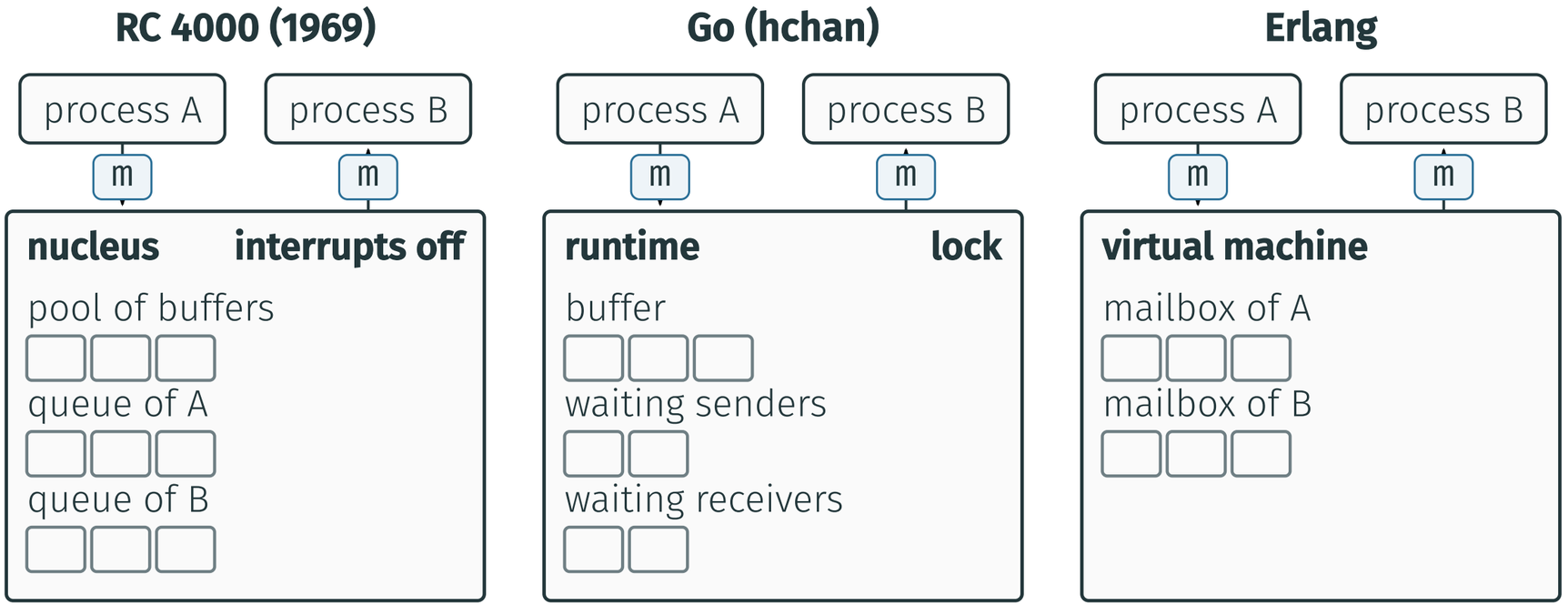
Both primitives went through the nucleus to the pool and the queue: the shared memory is still there, in one place. On the single processor the paper implies, a primitive runs with interrupts off, so in effect one lock guards that place.

The cut through the floor shows what the model hides: the nucleus, with a pool of buffers and one queue per process.

It copies m from A's memory into the buffer.

Wait message: the nucleus takes b1 from B's queue and gives B a second copy of m, with the sender and b1's identity; b1 stays out of the pool, held for the answer.

The shared memory inside the runtime



message passing on one node does not remove shared memory;
it hides it in one trusted place

The RC 4000: the nucleus holds a pool of buffers and one queue per process, the sorting room no process enters; a primitive runs with interrupts off, in effect one lock.

An Erlang mailbox is a queue per process, managed by the virtual machine.

A Go channel is a runtime structure, `hchan`, with a buffer, the queue of waiting senders, the queue of waiting receivers and a lock: the runtime of go1.27, not the specification, which says nothing of locks.

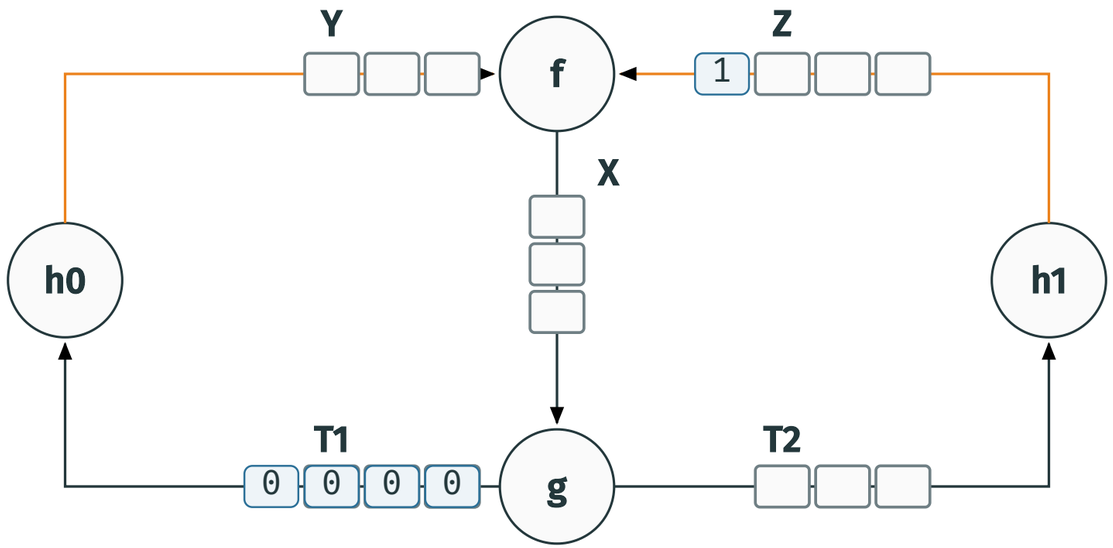
That place only has to be right once, as long as messages carry values and not shared pointers.

PART 4

Kahn process networks: determinism without choice

the model of 1974 and its result; two hello worlds; Unix pipelines; the three limits

Kahn's networks (1974)



f reads Y or Z as its own B says, never whichever channel holds a value

Each channel has exactly one writer and, in the form this lecture keeps, one reader: X is written by f and read by g, and by nobody else.

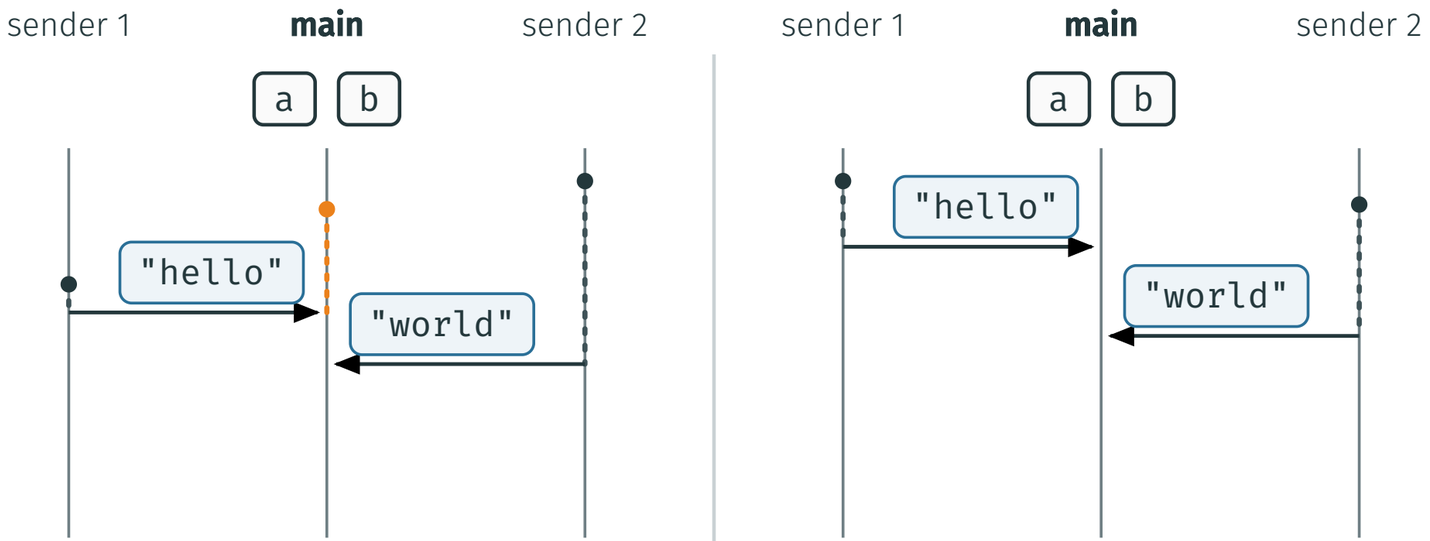
A read names one channel and waits until a value is there: f waits on Y while Z already holds a value.

Kahn's result, determinism. Under any fair schedule, one in which every process that can run eventually does, the network computes the same sequences on its channels; an unfair schedule computes a prefix of them.

A write never waits: g's sends on T1 land in the queue however slowly h0 reads, and the queue grows; the channel is an unbounded queue.

No choice among inputs anywhere: a process cannot ask whether something is on a channel, and cannot wait on whichever channel is ready first; which channel f reads next is fixed by its own program text.

Two senders and one receiver, with and without a contest



`hello world` every run

`hello world` every run

with one channel c the same senders print world hello or hello world

One run: world is received first and printed first; sender 1 waits until main receives again. This is the order nearly every run shows.

One channel per sender, main reading `a` then `b`: in one run main waits on `a` while world already waits on `b`; in the other, world waits. The waiting differs, the output never does.

Another run: hello first, and the output changes. With go1.27.1 at GOMAXPROCS 2 it happened 10 times in 3,000; the specification leaves the order among senders open.

The trade-off between determinism and responsiveness

> A contested communication buys responsiveness and costs determinism; no contest keeps determinism and costs waiting.

- Here the contest is a **merge**, two writers on one channel; a **guarded choice** in the program is the other way to let a contest in. Part 5 names both.
- The cost is a possibility, not a necessity: a choice whose branches lead to the same result keeps the result deterministic although the execution is not.
- A network without choice waits needlessly only where the channel it reads is empty while another is not.

Unix pipelines as Kahn networks

A plain shell pipeline is a Kahn network with bounded pipes: each pipe has one writer and one reader, and each program is deterministic, reading one input and writing one output. Five command lines, run on the lecture machine:

```
seq 1 100000 | grep 7 | sort -r | head -5
seq 5 | while read x; do sleep 0.$RANDOM; echo $x; done | awk '{print $1*$1}'
printf 'b\nZ\na\nB\nA\n' | LC_ALL=C sort
(dd if=/dev/zero bs=1k count=200 2>/dev/null; echo 'writer done' >&2) | (sleep 3; wc -c)
{ (sleep 0.0$RANDOM; echo hello) & (sleep 0.0$RANDOM; echo world) & wait; } | cat
```

The first printed the same five lines in 53 runs.

The second took between 1.35 and 2.38 seconds and printed `1 4 9 16 25` every time: a slow stage changes when, not what.

The third, under `LC_ALL=C`, printed `A B Z a b`; under `en_US.UTF-8` it printed `a A b B Z`: the environment is an input.

The fourth shows that a pipe is bounded: it holds 65,536 bytes on this machine, so the writer of 200 KiB finishes only after the reader wakes, at 3.02 seconds.

The last puts two writers on one pipe: in 20 runs, 14 printed `hello world` and 6 `world hello`. It is a merge, the first way out of Kahn's conditions.

The limits of a deterministic network

A deterministic network cannot:

1. serve whichever client asks first;
2. give up after a timeout;
3. react to whichever of two events happens first.

> **Each of the three is a decision that depends on timing, and a Kahn process cannot take one: its outputs are a function of the histories of its inputs, and a history records the values and their order, not the times at which they arrived. The three limits are what determinism costs.**

A merge that takes from whichever input has a value is accordingly not a Kahn process (Lee and Parks 1995). Real systems need all three, so competition has to be let back in.

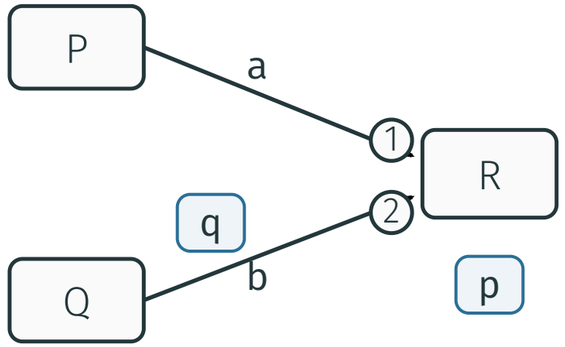
PART 5

Contested communications: three resolutions

where nondeterminism enters once nothing is shared; the lens of the session

Resolutions of a contested communication

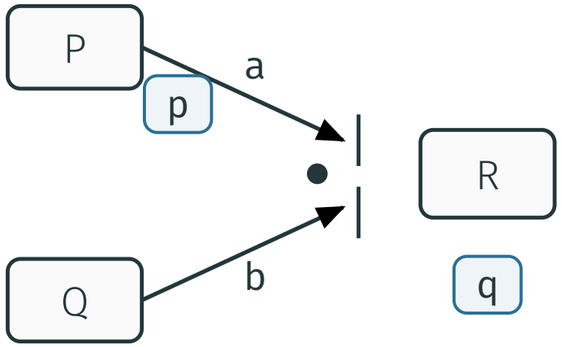
no contest
Kahn networks



one outcome: p, then q

```
<-a, then <-b
```

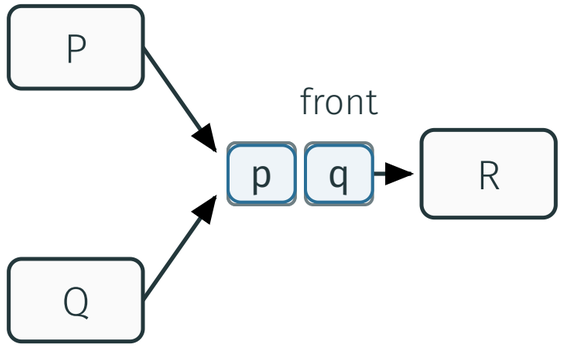
guarded choice
CSP, Go select



q first here; p first in another run

```
select on a and b
```

arrival order
RC 4000, actors, Erlang



q first here: it arrived first

```
receive Msg ->
```

No contest. One channel per sender, and R reads a, then b: Kahn's network. q, sent first, waits on b; there is one outcome, p then q, at the price of that wait.

Arrival order in a queue. Both messages go into R's queue, and R takes them from the front: the RC 4000, actors, Erlang. q arrived first and is served first; another run can bring p first: two outcomes.

A guarded choice in the program. R takes a partner that is ready: Hoare's guarded commands, Go's `select`. Here q is ready first and is taken; where both are ready, Go takes one by a uniform pseudo-random selection, so p can come first: two outcomes.

Sources of nondeterminism in message passing

Lee and Parks (1995) list five ways to add nondeterminism to a Kahn network:

1. a test for emptiness;
2. a process nondeterminate inside itself;
3. several writers on a channel;
4. several readers on a channel;
5. shared variables.

Message passing removes the last; a language adds a guarded choice.

Sources of nondeterminism in message passing

Once nothing is shared, nondeterminism can enter at four communication points, or inside a process:

1. many senders into one receiver, a **merge**;
2. many receivers on one channel, a **split**;
3. a **guarded choice** written by the programmer, asking which input is ready;
4. a **test for emptiness**, asking whether a message is there without waiting.

> In a program whose processes share nothing by any other route, whose messages carry values and whose runtime delivers them, nondeterminism can enter only where several parties compete for one communication point, where a process tests whether a communication is possible, or inside a process. A contest allows a different outcome and does not force one.

PART 6

The RC 4000 nucleus: two resolutions in one machine

the four primitives; requests in arrival order; replies by identification

The communication primitives of the RC 4000 nucleus

```
client:
  buf := send message(server, m)
      -- returns at once
  ...other work...
  answer := wait answer(buf)
```

```
server:
  loop
    (sender, m, buf) := wait message()
    ...serve m...
    send answer(buf, result)
```

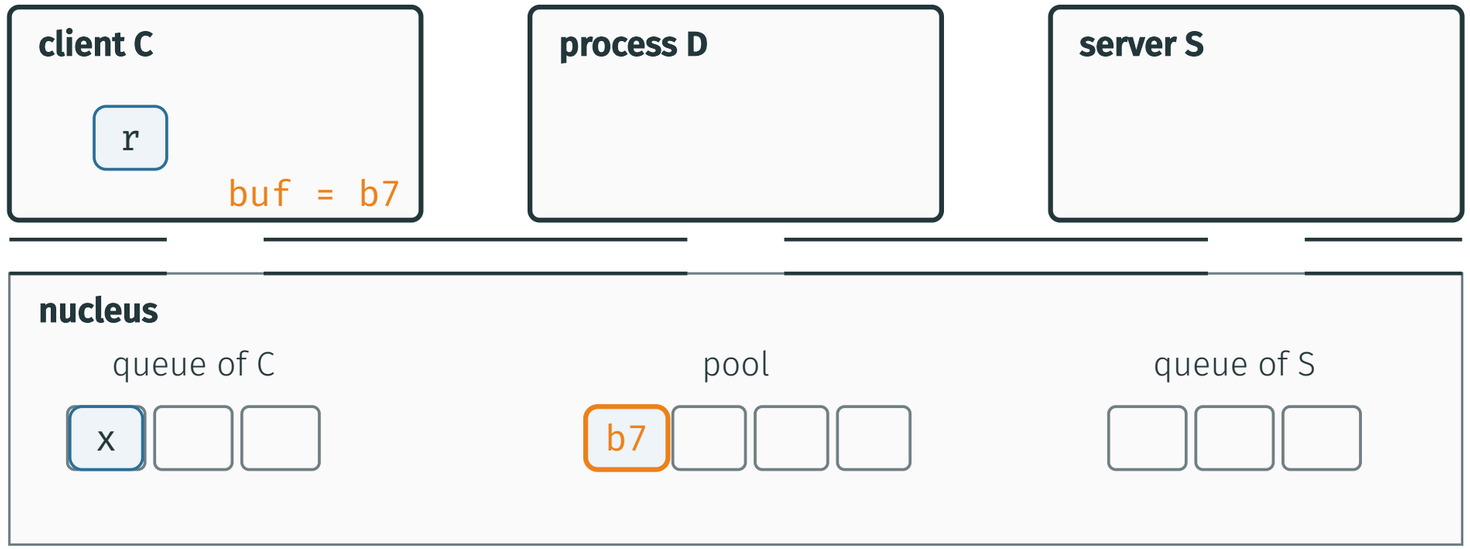
The client calls *send message*, naming the server; within its limit of outstanding messages, the call returns at once with the identity of the buffer used, and the client goes on with other work.

It serves the request and calls *send answer*, which copies the answer into the same buffer; the server continues at once.

The server waits in *wait message* until a message is in its queue, and receives the sender's name, the message and the buffer.

The client calls *wait answer* on its buffer and waits until the answer is there; the buffer then goes back to the pool.

One conversation on the RC 4000



C sends m to S: the nucleus takes buffer b7 from the pool and links it behind d; C continues at once, holding b7's identity.

C calls wait answer(b7) before the answer exists: C waits.

Then b7: a copy of m goes into S's memory, and b7 stays in the nucleus, held for the answer. Arrival order has settled the contest between D and C.

C's wait answer(b7) takes b7 past x, by its identity, which the nucleus checks: a copy of the answer goes into C's memory. Nobody competed for the reply.

While C does other work, D sends x to C: another buffer lands in C's queue.

S calls wait message, first come, first served: d first, and a copy of d goes into S's memory.

S sends its answer: the nucleus puts it in the same buffer, now b7 · r, which joins C's queue behind x, an unrelated message.

The conversation is over: b7 goes back to the pool.

Requests and replies

Requests	A send names a process, which has one queue; messages from every sender land in it in arrival order, and *wait message* serves them first come, first served. When several clients compete for one server, arrival order in the queue resolves the contest: the mailbox family's resolution, four years before Hewitt's actors.
Replies	The answer travels back in the same buffer, and the client waits for that buffer by its identity. Nobody competes for a reply, because the client waits for one identified item and not for the next thing in its queue: the no-contest resolution, obtained by identifying the conversation.
Today	Go's reply channel carried in the request (part 7); Erlang's reference matched by a selective receive (block 2). The nucleus checks the identity; Go's and Erlang's programs keep it by convention.

PART 7

Channels: synchronous communication and guarded choice

CSP (1978), its channel form (1985), occam and Go

Communicating sequential processes (Hoare, 1978)

Hoare proposes input and output as basic primitives of programming, and parallel composition of communicating sequential processes as a fundamental way to structure programs. Four of its choices define the language as this story reads it.

1. **No shared variables.** Processes in a parallel command are disjoint.

2. **Named partners.** An input names its source process and an output its destination: `X?(x, y)` in one process and `DIV!(3*a+b, 13)` in the other together act as the assignment `(x, y) := (3*a+b, 13)`.

3. **No buffering.** "There is no automatic buffering." Whichever side is ready first waits, and "such delay is invisible to the delayed process".

4. **An input fails if its source has terminated, and an output fails if its destination has.** The paper's own statement, page 669.

Guarded choice

```
*[ n < 10; producer?x →
       buffer(n) := x; n := n + 1
 □ n > 0;  consumer?more() →
       consumer!buffer(0); shift
 ]
```

Hoare's bounded buffer, sketched: the process offers two communications and takes whichever partner turns up.

A guard may end in an input command, and the guarded branch runs only if the named source is ready to output: the producer while the buffer has room, the consumer while it holds stock.

The branch after the arrow runs once the communication has happened: the item stored and counted, or the first item sent and the buffer shifted.

When several guards are ready, exactly one is chosen, arbitrarily: the language selects, at this one visible point.

This is the guarded choice in the program: the programmer marks one visible point, the language selects among the partners ready there, and everything else stays deterministic.

Kahn's networks and CSP compared

Kahn's networks (1974) and CSP (1978) share three things; Hoare listed the differences himself (1978, §7.7), and only one of them decides the values a program computes.

- **No shared memory.** A process affects another only through the value it sends.
- **Partners fixed by the program text.** The writer of a channel in Kahn's networks; the named process in CSP of 1978, where no channel object exists.
- **A read that waits.** In both, a read waits until a value has arrived.

Kahn's networks and CSP compared

Kahn's networks	CSP (1978)	Decides the values computed
no choice between sources of input	a guarded choice among ready partners	yes
output buffered to any degree	no buffering: the handshake	no: waiting and deadlock
output fanned out to any number of readers	one partner per exchange	no: the same value for each
processes run forever	the parallel command terminates	no: a history ends

Hoare's list, 1978, §7.7; the 1985 book repeats the first three differences (§7.3.3). Of the four, only the first decides the values computed.

Kahn's networks and CSP compared

- **Buffering** decides waiting, deadlock and the sender's knowledge; no value changes.
- **Fan-out**: a value copied to several readers is the same value for each.
- **Termination** ends a history without changing its values, unless a process acts on a partner's termination.
- **Choice** does change the values: a guarded choice among ready partners is exactly what Kahn's rules forbid.

> A CSP program with deterministic processes, no alternative command whose guards can succeed together, one writer and one reader per channel, and no process acting on a partner's termination computes as a Kahn network whose channels hold nothing. Choice, not synchrony, separates the two models in their results.

Reasons for synchronous communication

Four reasons for the unbuffered send come from different sources; only the second is Hoare's own, from the paper of 1978.

1. **Provability.** A communication is one event shared by two processes; with a buffer there is a send event, a receive event, and hidden queue state between them. The algebra of CSP that followed, Hoare's 1985 book, is built on the shared event: a later reading, not a reason given in 1978.

2. **No arbitrary buffer size.** Hoare chose size zero and let programmers write buffers as processes, for two reasons: buffering is less realistic on separate processors, and it can be built from the primitives where needed. He concedes the symmetric point, that synchronization could be built from buffered commands (1978, page 675).

Reasons for synchronous communication

3. **Knowledge.** After a synchronous send, the sender knows the receiver has the value, and can make assertions about the receiver's state (Andrews and Schneider, 1983, page 27).

4. **Backpressure.** A fast producer cannot run ahead and fill memory. This is true of any bounded buffer, zero included, so it is a reason for bounding rather than for size zero, and it is not in the 1978 paper.

Reason	Provability	No arbitrary buffer size	Knowledge	Backpressure
Source	a later reading	Hoare 1978	Andrews and Schneider 1983	a later reading

From named partners to channels

```
B :: *[ x: integer; P?x → C!x ]
```

1978: B names its partners, P and C.

In 1978 style, B names its partners, P and C. To reuse B between two other processes, the programmer must rewrite B.

The handshake is unchanged; only the naming changed. occam (1983) and Hoare's 1985 book made named synchronous channels the norm, and that channel version is what Go took up.

```
B(in, out) = in?x → out!x → B(in, out)
```

1985: B names its own two ends.

With channels, the parameters `in` and `out` are the channels and the right-hand side is B's behavior; whoever assembles the system plugs real channels into them: `B(left, mid) ‖ B(mid, right)` joins two buffers through `mid`.

Go's rules as the specification states them

- With capacity zero, a channel is unbuffered and "communication succeeds only when both a sender and receiver are ready". With capacity *n*, a send proceeds if the buffer is not full and a receive if it is not empty.
- A channel is a first-in-first-out queue for one sender and one receiver; the specification states no order among concurrent senders.
- A send on a closed channel panics; a send or receive on a `nil` channel blocks forever.
- A `select` chooses among cases that can proceed by "a uniform pseudo-random selection". With none ready, it takes `default` if there is one; otherwise it blocks.

Go's `select` allows send cases as well as receive cases; Hoare's 1978 language allowed only input guards. That difference matters in session 12.

The reply channel carried in the request

Go

reply_server.go ↗

```
type req struct {
    n     int
    reply chan<- int // send-only
}

func counter(reqs <-chan req) {
    total := 0
    for r := range reqs { // one request at a time
        total += r.n
        r.reply <- total  // the answer goes back
    }
}

// a client
reply := make(chan int)
reqs <- req{5, reply}
fmt.Println(<-reply)
```

The request carries its reply channel, typed send-only, `chan<- int`.

The answer goes back on the channel the request carried: the identified reply.

It sends the request with the channel inside.

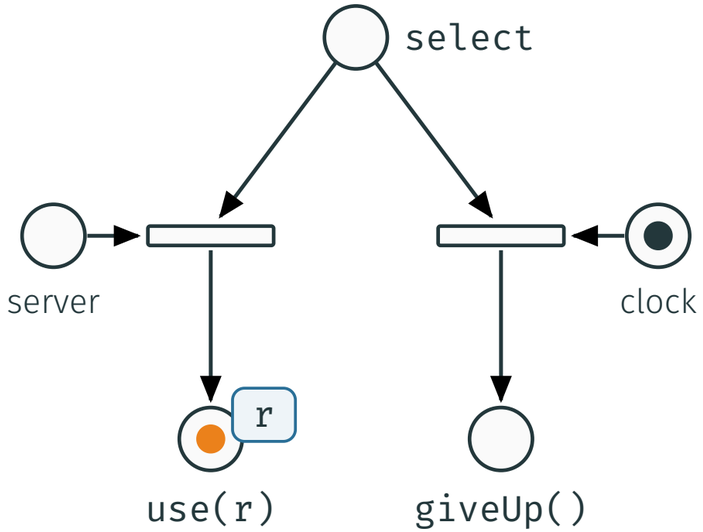
The server takes one request at a time from `reqs`: the queue of requests is the nucleus's queue of the server.

A client makes a fresh reply channel, the identity of this conversation.

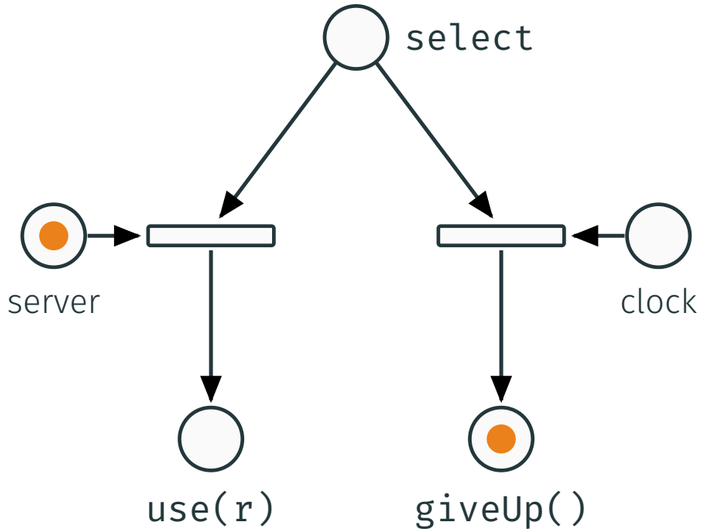
It receives on its own channel. Only the client reads it, by convention: the type stops the server from reading it, and nothing stops another goroutine that holds it. The nucleus checked the identity; here the program keeps it.

A timeout as one more branch of the choice

both ready: reply case taken



both ready: timeout case taken



In one run the server is ready first: its bar fires, r crosses it, and the goroutine goes on into `use(r)`.

When both are ready as the `select` runs, it takes one case by a uniform pseudo-random selection; the two runs take different branches. In the run that takes the timeout, the server's send on the unbuffered reply channel stays blocked, since nothing receives it.

In another run the clock fires first: the timeout branch runs, `giveUp()`.

Deadlock by waiting

Go

send_completion.go ↗

```go
a, b := make(chan int), make(chan int)
go func() { a <- <-b }()
fmt.Println(<-a)
```

```
fatal error: all goroutines are asleep - deadlock!
```

A missing partner: `main` receives on `a`, the goroutine waits on `b` before it would send on `a`, and nobody sends on `b`.

Go

cycle_min.go ↗

```go
toMain, toOther := make(chan int), make(chan int)
go func() { toMain <- 101; fmt.Println(<-toOther) }()
toOther <- 1
fmt.Println(<-toMain)
```

```
fatal error: all goroutines are asleep - deadlock!
```

A cycle: each goroutine sends first and receives afterwards; on unbuffered channels each send waits for a receive the other reaches only after its own send. This cycle deadlocks in every run.

> A deadlock needs no lock, only waiting: a lock is one thing a thread can wait for, and any operation that waits can play the part a lock played in session 7.

Buffer capacity: waiting, deadlock and the sender's knowledge

the second parameter; the cycle at capacities 0, 1 and 2; capacity and the semaphore count; the two summaries

The capacity of a channel

Capacity	Systems	What a send means
0	CSP (1978, 1985), occam, Go's unbuffered channel	the receiver has the value; the smallest cycle of sends already deadlocks
k	Unix pipes (65,536 bytes here); `make(chan T, k)`; the RC 4000's pool, a limit per sending process	the value is queued and the sender goes on; a full pipe or Go channel makes the sender wait, which is backpressure
unbounded	Kahn (1974); actors and Erlang's mailboxes (block 2)	a sender never waits and learns nothing of the receiver; no backpressure, and memory grows

> In a network without choice or test for emptiness and with one writer and one reader per channel, buffer size does not decide the results, only waiting, deadlock and what the sender knows; in a program with choice or with a merge, it can also change which outcomes are possible.

Asynchrony and nondeterminism

Asynchrony	a send that does not wait for its receiver: a matter of buffering
Nondeterminism	once nothing is shared, a matter of contests, tests for emptiness and processes nondeterminate inside

- The two are set independently: an unbounded Kahn network is asynchronous and deterministic; a CSP program with a guarded choice is synchronous and can be nondeterministic.
- Where a contest exists, capacity can still change which outcomes are possible.

The cycle at capacities 0, 1 and 2

Go

cycle.go ↗

```go
// go run cycle.go <capacity> <sends each>
// every receive prints its value; main then
// waits for the goroutine and prints finished
toMain := make(chan int, k)
toOther := make(chan int, k)
go func() {
    for i := 1; i <= sends; i++ { toMain <- 100 + i }
    for range sends { <-toOther }
}()
for i := 1; i <= sends; i++ { toOther <- i }
for range sends { <-toMain }
```

1. capacity 0, one send each
2. capacity 1, one send each
3. capacity 1, two sends each
4. capacity 2, two sends each

```
$ go run cycle.go 0 1
fatal error: all goroutines are a
```

```
$ go run cycle.go 1 1
other received 1
main received 101
finished
```

```
$ go run cycle.go 1 2
fatal error: all goroutines are a
```

```
$ go run cycle.go 2 2
other received 1
other received 2
main received 101
main received 102
finished
```

Capacity decided who waits and whether the program deadlocks; it computed no other value. The missing partner stays at every capacity, since a receive on a channel nobody writes waits forever. A buffer can break a cycle; it cannot supply a missing partner.

The effects of buffer capacity

1. determinism and the buffer
2. size zero and the meaning of a choice
3. the senders' contest, moved
4. eventual delivery and fairness
5. backpressure against load shedding

Determinism comes from the absence of competition, not from the buffer. A Kahn network gives the same values with any buffer size; with buffers that are too small it can deadlock where unbounded buffers would not. The pipelines of part 4 with one writer per pipe are an instance.

Size zero changes what a choice means. With a handshake, a `select` picks among partners ready at that instant, and the sender knows the value arrived. With buffers, sends finish early, more interleavings become possible, and the sender knows less.

A bounded buffer moves the senders' contest. When it is full, senders compete for the next free slot instead of for the receiver. In Go, a send case in a `select` is ready on a buffered channel when there is room, on an unbuffered one when a receiver is waiting.

Eventual delivery with no bound on delay is where actors get unbounded nondeterminism. The source is the fairness assumption and not the buffer: senders never wait and delivery is assumed eventual, so outcomes can become infinite (block 2).

Backpressure against load shedding. Unbounded buffers give no backpressure: a slow receiver goes unnoticed while its mailbox grows. A bounded buffer that blocks the sender gives backpressure, paid for in blocking bugs; one that drops the message or returns an error sheds load without blocking.

Buffer capacity and the semaphore count

Capacity decides which interleavings of sends and receives can occur; its exact counterpart in session 7 is the initial count of a counting semaphore, not the lock. A lock forbids two threads to be in one region at the same time; a capacity forbids one side to run more than *k* steps ahead of the other.

Capacity of a channel (session 8)	**Semaphores (session 7)**
0, the handshake	a rendezvous: one semaphore signals that a value is there, a second that it was taken, and a third keeps a second sender out; Java's `SynchronousQueue`, which has no internal capacity, "not even a capacity of one"
k	Dijkstra's bounded buffer (1965): a semaphore of queuing portions that starts at 0, a semaphore of empty positions that starts at the buffer's size, and a binary semaphore that guards the array; the producer ran at most *k* items ahead
unbounded	Dijkstra's first buffer: the semaphore of queuing portions and the binary semaphore, with no count of empty positions, so the producer never waits

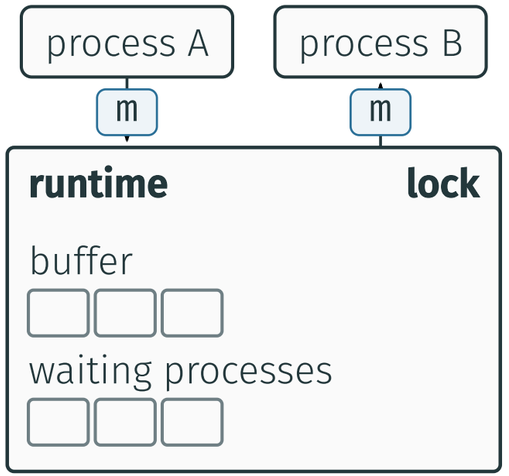
Buffer capacity and the semaphore count

> More synchronization always leaves fewer interleavings. It leaves fewer observable outcomes only where a contest exists, a merge, a split or a choice, or for threads a race the constraint covers; in a Kahn network every value produced is the same at every capacity, a small one only cutting the output short.

The number	fixed at creation in both cases; the code that uses it only sends and receives, or acquires and releases
A reading of it	`len(ch)` for a Go channel, `availablePermits()` for a Java semaphore: out of date by the next statement, since another thread can change it at once
A decision on it	a test for emptiness in the sense of part 5, whose outcome depends on timing; Dijkstra's semaphore has no such operation, P and V being the only way in

Summary: the trusted place and the marked points

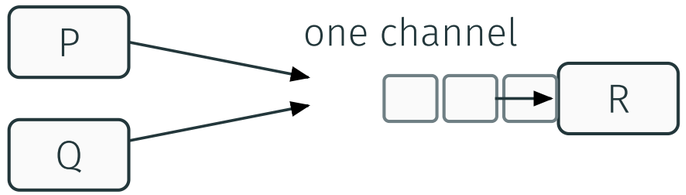
one trusted place



interleavings of accesses, under one lock

a few marked points

a merge



a guarded choice



contests at points the program marks

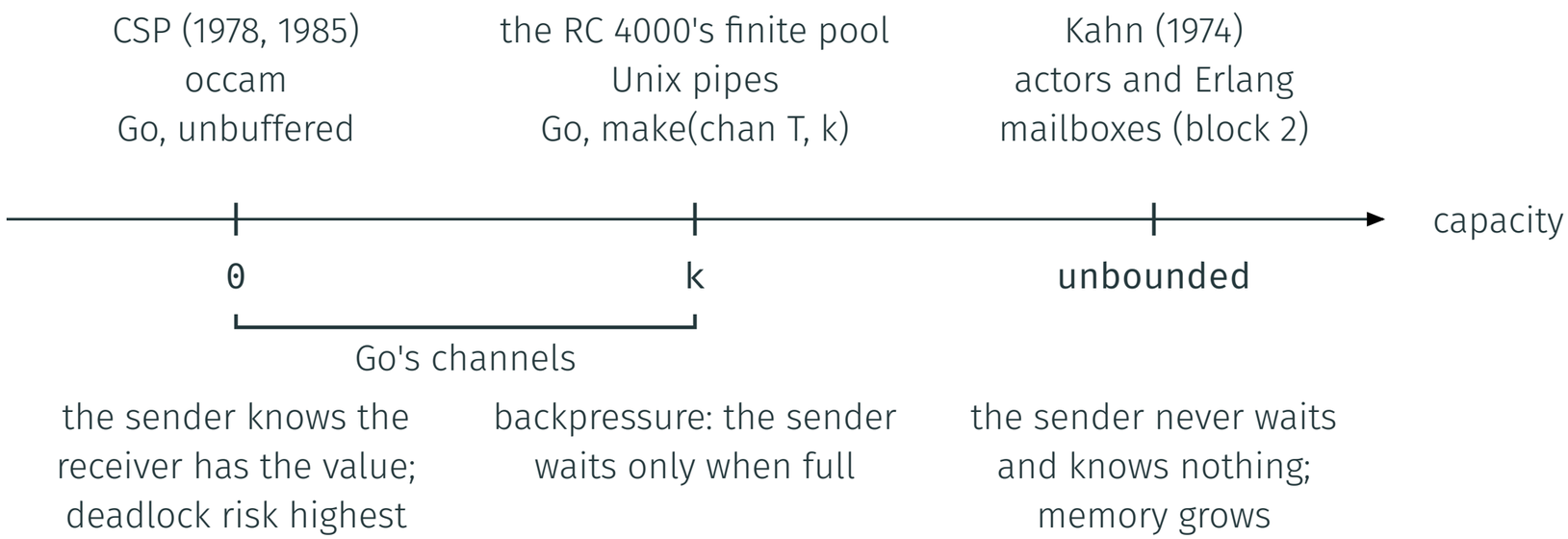
message passing does not remove interleavings; it moves them,
into one trusted place and onto a few marked points

The interleavings of accesses to shared memory go into the one trusted place underneath, the buffer and the lock that orders them.

Determinism comes from the absence of contest, not from the buffer: no contest leaves one outcome at the price of waiting; a guarded choice or arrival order leaves two, at one visible place.

The scheduler's choices that can reach the results go to the few marked points where the program admits a contest: a merge, a guarded choice (CSP, Go), arrival order in a queue (the RC 4000, the actors).

Summary: the axis of buffer capacity



in a Kahn network the values computed are the same everywhere on this axis

At 0, the handshake: CSP (1978, 1985), occam, Go's unbuffered channel.

Unbounded: Kahn's networks, and the actors and Erlang mailboxes of block 2.

In a Kahn network the values computed are the same everywhere on this axis; capacity decides who waits, when, and how many interleavings remain.

At a finite k: the RC 4000's pool, Unix pipes, `make(chan T, k)`; Go's channels span 0 to k.

At 0 the sender knows the receiver has the value, and the smallest cycle deadlocks; at k backpressure, the sender waiting only when full; unbounded, no waiting, no knowledge, and memory that grows.

Block 2 takes the request half of the RC 4000's conversation to its full form, the mailbox: a send that never waits, an arrival order left free, a delivery assumed.